

HY360

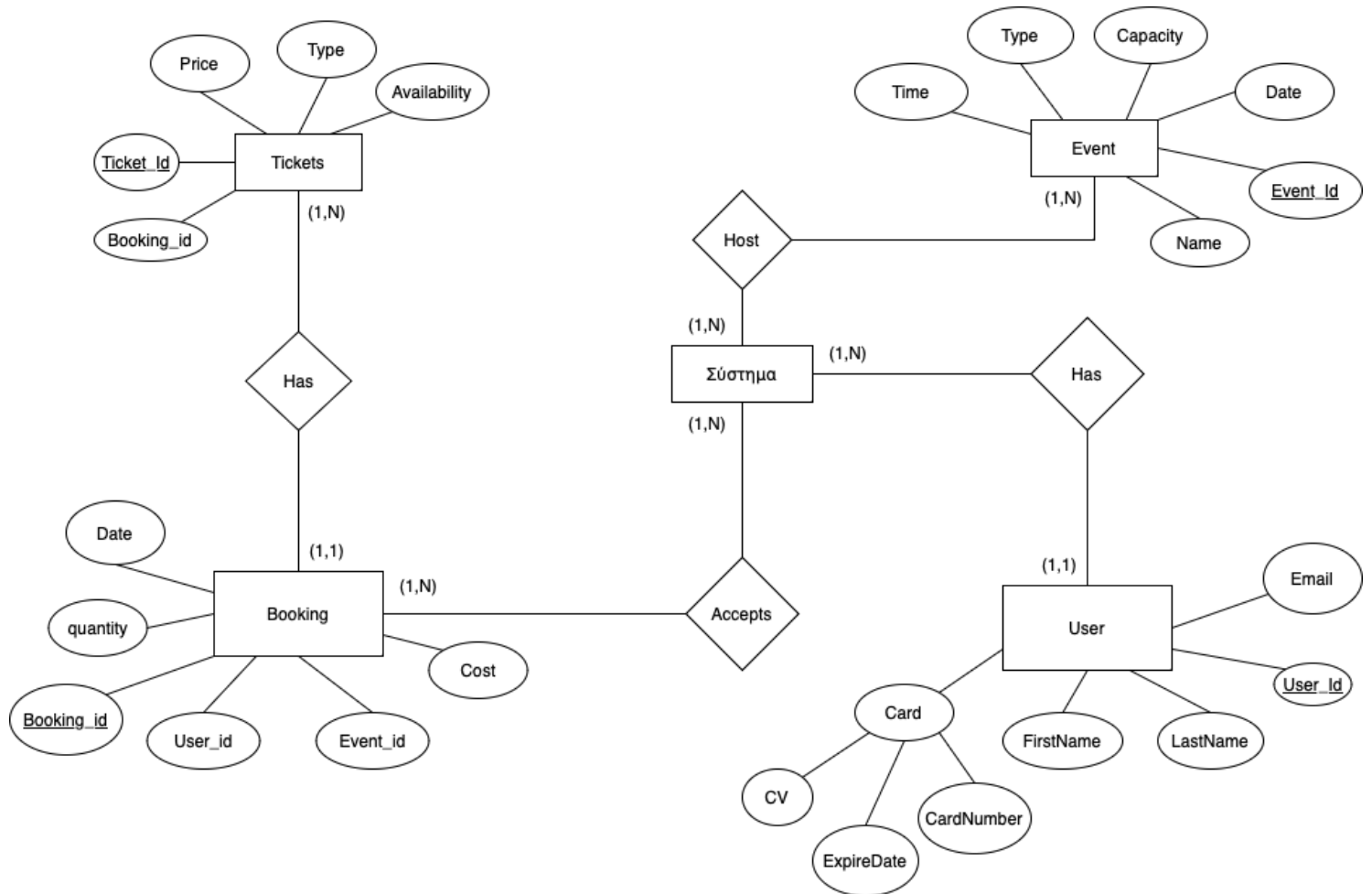
PROJECT

ANTPEΑΣ ΣΙΝΑΝΗΣ csd5150

ΑΛΑΤΣΑΚΗ ΜΑΡΙΑ ΦΩΤΕΙΝΗ csd4584

ΓΙΑΝΝΗΣ ΚΟΡΣΑΒΒΑΣ csd4913

- ΔΙΑΓΡΑΜΜΑ ΟΝΤΟΤΗΤΩΝ - ΣΧΕΣΕΩΝ E-R



- ONTOTHTES KAI GNΩPISΜATA ONTOTHTΩN

User

user_id	INT	AUTO_INCREMENT	PRIMARY KEY
card_number	INT	NOT NULL	
firstName	VARCHAR(20)	NOT NULL	
lastname	VARCHAR(20)	NOT NULL	
Email	VARCHAR(30)		
expiry_date	DATE		
cvv	INT		

EVENT

event_id	INT	AUTO_INCREMENT	PRIMARY KEY
event_date	DATE		
name	VARCHAR(20)		
Time	DATETIME		
type	VARCHAR(20)		
Capacity	INT		

TICKETS

ticket_id	INT	AUTO_INCREMENT	PRIMARY KEY
Available	BOOLEAN		
Price	INT		
Type	VARCHAR(20)		

BOOKING

booking_id	INT	AUTO_INCREMENT	PRIMARY KEY
user_id	INT		
event_id	INT		
Quantity	INT		
Date	DATE		
Cost	INT		

- Επεξηγήσεις γνωρισμάτων οντοτήτων

- i. Η οντότητα **User** περιέχει το user_id που είναι το χαρακτηριστικό id του χρήστη (κάθε χρήστης έχει ένα μοναδικό χαρακτηριστικό id), το ονοματεπώνυμο του, το email και το card που περιέχει τα στοιχεία της κάρτας όπως, card_number, expiry_date και env.
- ii. Η οντότητα **Event** περιέχει το χαρακτηριστικό id, event_id, το event_date που είναι η ημερομηνία του event καθώς και το time που είναι η ώρα που θα γίνει. Επίσης, το name, το type που είναι το τι event είναι καθώς και capacity που είναι το πόσα άτομα μπορεί να δεχτεί το event.
- iii. Η οντότητα **Tickets** περιέχει το χαρακτηριστικό id, ticket_id, το type που είναι τι τύπος εισιτηρίου είναι δηλαδή, VIP, γενική είσοδος κλπ, το available που δείχνει αν το εισιτήριο είναι διαθέσιμο και την τιμή του price.
- iv. Η οντότητα **Booking** περιέχει το χαρακτηριστικό id, booking_id, το user_id του χρήστη που έχει κάνει την κράτηση, το event_id του event που έχει κάνει κράτηση ο χρήστης, τον αριθμό εισιτηρίων που έχει κάνει κράτηση, το date που είναι η ημερομηνία της κράτησης και το κόστος.

- Πρωτεύοντα κλειδιά

- i. Η οντότητα **User** έχει ως πρωτεύον κλειδί το user_id που είναι το χαρακτηριστικό id του χρήστη (κάθε χρήστης έχει ένα μοναδικό χαρακτηριστικό id) και δίνεται από το σύστημα κατά την εγγραφή του χρήστη στο σύστημα.
- ii. Η οντότητα **Event** έχει ως πρωτεύον κλειδί το event_id, που είναι το χαρακτηριστικό id του event (κάθε event έχει ένα μοναδικό χαρακτηριστικό id) και δίνεται από το σύστημα κατά την δημιουργία ενός νέου event.
- iii. Η οντότητα **Tickets** έχει ως πρωτεύον κλειδί το ticket_id, που είναι το χαρακτηριστικό id του ticket (κάθε ticket έχει ένα μοναδικό χαρακτηριστικό id).
- iv. Η οντότητα **Booking** έχει ως πρωτεύον κλειδί το booking_id, που είναι το χαρακτηριστικό id του event (κάθε booking έχει ένα μοναδικό χαρακτηριστικό id).

- Περιορισμοί πληθικότητας
Οι περιορισμοί πληθικότητας βρίσκονται πάνω στο διάγραμμα E-R.

- Σχεσιακό Μοντέλο

Tickets

<u>Ticket_id</u>	Price	Type	Availability	<u>Booking_Id</u>
------------------	-------	------	--------------	-------------------

Bookings

Date	Quantity	<u>Booking_id</u>	User_id	Event_id	Cost	Ticket_id
------	----------	-------------------	---------	----------	------	-----------

Users

CV	ExpireDate	CardNumber	FirstName	LastName	<u>User_id</u>	Email
----	------------	------------	-----------	----------	----------------	-------

Events

Time	Type	Capacity	Date	<u>Event_id</u>	Name
------	------	----------	------	-----------------	------

- Συναρτησιακές Εξαρτήσεις
 - booking_id -> date, quantity, user_id, event_id, cost, ticket_id
(Booking)
 - user_id -> cv, expireDate, cardNumber, firstName, lastName, email
(Users)
 - event_id -> time, type, capacity, date, name (Events)
 - ticket_id, booking_id -> price, type, availability

- Τρίτη Κανονική Μορφή - 3NF

1. Πρώτη Κανονική Μορφή (1NF)

Η βάση δεδομένων είναι **1NF** καθώς:

- Όλα τα γνωρίσματα είναι ατομικά (δεν περιέχουν πλειότιμα ή σύνθετα δεδομένα).
- Κάθε σχέση έχει μοναδικό πρωτεύον κλειδί και δεν υπάρχουν επαναλαμβανόμενες ομάδες δεδομένων.

Δηλαδή:

- Στη σχέση **Tickets**, κάθε εγγραφή περιέχει μοναδικό Ticket_Id, ενώ τα γνωρίσματα όπως Price, Type και Availability είναι ατομικά.
- Αντίστοιχα, σε όλες τις άλλες σχέσεις, τα γνωρίσματα είναι σωστά δομημένα.

2. Δεύτερη Κανονική Μορφή (2NF)

Η βάση δεδομένων είναι **2NF** διότι:

- Όλες οι σχέσεις είναι σε 1NF.
- Δεν υπάρχουν μερικές συναρτησιακές εξαρτήσεις. Δηλαδή, κάθε μη πρωτεύον γνώρισμα εξαρτάται πλήρως από το πρωτεύον κλειδί της κάθε σχέσης.

Δηλαδή:

- Στη σχέση **Booking**, το Date, το Cost, και το Quantity εξαρτώνται πλήρως από το Booking_Id και όχι από κάποιο υποσύνολό του.
- Αντίστοιχα, στη σχέση **User**, τα γνωρίσματα όπως FirstName, LastName, και Email εξαρτώνται πλήρως από το User_Id.

3. Τρίτη Κανονική Μορφή (3NF)

Η βάση δεδομένων πληροί την **3NF** καθώς:

- Είναι σε 2NF.
- Δεν υπάρχουν μεταβατικές εξαρτήσεις. Δηλαδή, κάθε μη πρωτεύον γνώρισμα εξαρτάται άμεσα από το πρωτεύον κλειδί και όχι έμμεσα μέσω κάποιου άλλου μη πρωτεύοντος γνωρίσματος.

Δηλαδή:

- Στη σχέση **User**, το cv, το expireDate, το cardNumber, το firstName, το lastName και το email εξαρτώνται άμεσα από το User_Id.
- Στη σχέση **Booking**, δεν υπάρχει γνώρισμα που να εξαρτάται έμμεσα από το Booking_Id.

Επομένως, η βάση δεδομένων μας βρίσκεται σε Τρίτη Κανονική Μορφή (3NF).

- Εντολές SQL για τη δημιουργία των πινάκων

```
CREATE TABLE tickets(
    ticket_id INT not NULL AUTO_INCREMENT,
    available BOOLEAN,
    price INT not NULL,
    type VARCHAR(20),
    PRIMARY KEY (ticket_id)
);
```

```
CREATE TABLE Bookings(
    booking_id INT not NULL AUTO_INCREMENT,
    user_id INT,
    event_id INT,
    quantity INT,
    date DATE,
    cost INT,
    PRIMARY KEY (booking_id)
);
```

```
CREATE TABLE users(
    user_id INT INTEGER not NULL AUTO_INCREMENT,
    card_number INT,
    first_name VARCHAR(20) not NULL,
    last_name VARCHAR(20) not NULL,
    email VARCHAR(20),
    expiry_date DATE,
    cvv INT not NULL,
    PRIMARY KEY (user_id)
);
```

```
CREATE TABLE events(
    event_id INT not NULL AUTO_INCREMENT,
    event_date DATE,
    name VARCHAR(20),
    time DATETIME,
```

```

type VARCHAR(20),
seat INTEGER,
vip INTEGER,
regular INTEGER,
income INTEGER DEFAULT,
capacity INT AS(vip+regular+seat) STORED,
capacity INT DEFAULT 0,
PRIMARY KEY (event_id)
);

```

- Ερωτήσεις προς την ΒΔ με SQL

Για την Δημιουργία νέας εκδήλωσης στο σύστημα

```

INSERT INTO `events`(`event_date`, `name`, `time`, `vip`, `seat`, `regular`,
`type`) VALUES (?, ?, ?, ?, ?, ?, ?)

```

Για την εγγραφή νέου πελάτη

```

INSERT INTO `users`(`card_number`, `first_name`, `last_name`, `email`,
`expiry_date`, `cvv`) VALUES (?, ?, ?, ?, ?, ?)

```

Για αναζήτηση διαθέσιμων θέσεων

```

SELECT * FROM events

```

Για κράτηση εισιτηρίων

```

"INSERT INTO bookings (user_id, event_id, quantity, date, cost) VALUES
(?, ?, ?, ?, ?)";

```

```

UPDATE events SET vip = vip - ?, regular = regular - ?, seat = seat - ?, income =
income + ? WHERE event_id = ?;

```

```

INSERT INTO tickets (available, price, booking_id, type) VALUES (?, ?, ?, ?)

```

Για ακύρωση κράτησης

```

SELECT `user_id`, `event_id` FROM `bookings` WHERE ?
"SELECT `type` FROM `tickets` WHERE `booking_id` = ?";
DELETE FROM `tickets` WHERE `booking_id` = ?
DELETE FROM `bookings` WHERE `booking_id` = ?

```

Για ακύρωση εκδήλωσης

```

"SELECT booking_id FROM bookings WHERE event_id = ?";DELETE FROM
bookings WHERE event_id = ?;

```

και στην περίπτωση που είναι χρέωση κράτησης,

```

UPDATE events SET vip = vip + ?, regular = regular + ?, seat = seat + ?, income =
income - ? WHERE event_id = ?

```

αλλιώς,

```

UPDATE events SET vip = vip + ?, regular = regular + ?, seat = seat + ?, ncome =
income - ? WHERE event_id = ?

```

Κατάσταση διαθέσιμων και κρατημένων θέσεων ανά εκδήλωση.


```
SELECT e.event_id, e.capacity AS available_tickets, COALESCE(SUM(CASE WHEN
t.type = 'VIP' THEN 1 ELSE 0 END), 0) AS booked_vip, COALESCE(SUM(CASE
WHEN t.type = 'Regular' THEN 1 ELSE 0 END), 0) AS booked_regular,
COALESCE(SUM(CASE WHEN t.type = 'Seat' THEN 1 ELSE 0 END), 0) AS
booked_seat FROM events e LEFT JOIN bookings b ON e.event_id = b.event_id
LEFT JOIN tickets t ON b.booking_id = t.booking_id GROUP BY e.event_id,
e.capacity
```

Έσοδα από πωλήσεις ανά εκδήλωση.

```
SELECT e.event_id, e.name, e.event_date, SUM(b.cost) AS income FROM events e
JOIN bookings b ON e.event_id = b.event_id GROUP BY e.event_id, e.name,
e.event_date ORDER BY income DESC
```

Δημοφιλέστερη εκδήλωση βάσει κρατήσεων.

```
SELECT e.event_id, e.name AS event_name, e.event_date, SUM(b.quantity) AS
total_reservations FROM events e JOIN bookings b ON e.event_id = b.event_id
GROUP BY e.event_id, e.name, e.event_date ORDER BY total_reservations DESC
LIMIT 1
```

Εκδήλωση με τα περισσότερα έσοδα σε ένα χρονικό εύρος.

```
SELECT event_id, name AS event_name, event_date, income FROM events WHERE
event_date BETWEEN ? AND ? ORDER BY income DESC LIMIT 1
```

Προβολή κρατήσεων ανά χρονική περίοδο.

```
SELECT YEAR(e.event_date) AS event_year, COUNT(b.booking_id) AS
total_bookings FROM bookings b JOIN events e ON b.event_id = e.event_id GROUP
BY YEAR(e.event_date) ORDER BY event_year
```

Τα συνολικά έσοδα από την πώληση VIP ή γενικών εισιτηρίων ανά εκδήλωση ή συνολικά

```
SELECT e.event_id, e.name AS event_name, SUM(CASE WHEN t.type = 'VIP' THEN
30 * 1 ELSE 0 END) AS vip_income, SUM(CASE WHEN t.type = 'regular' THEN 10 * 1
ELSE 0 END) AS regular_income, SUM(CASE WHEN t.type = 'seat' THEN 20 * 1
ELSE 0 END) AS seat_income, SUM(CASE WHEN t.type = 'VIP' THEN 30 * 1
WHEN t.type = 'regular' THEN 10 * 1 WHEN t.type = 'seat' THEN 20 * 1 ELSE 0 END
AS total_income FROM bookings b JOIN tickets t ON b.booking_id = t.booking_id
JOIN events e ON b.event_id = e.event_id GROUP BY e.event_id, e.name ORDER BY
total_income DESC
```